

Apostila Didática:

Introdução à Robótica com Sparki

Projeto Edubot - Universidade de Brasília (UnB)

Resumo—Essa apostila foi criada pela equipe do Edubot para te ajudar a desvendar o *Sparki*, o robzinho da Arcbotics. Nosso objetivo é mostrar que programar sensores e motores pode ser simples, divertido e acessível para qualquer estudante.

I. Quem Somos Nós?

O Edubot é um Projeto de Extensão da Universidade de Brasília (UnB), apoiado pelo capítulo RAS (*Robotics and Automation Society*) da IEEE.

Nosso objetivo é incentivar jovens a conhecer as áreas de Tecnologia, Programação e Engenharia por meio de oficinas práticas em escolas públicas do Distrito Federal.

Instagram: @projetoedubot

II. Introdução

O Sparki é um robô educacional equipado com sensores de distância, luz, motores e diversos outros recursos.

Aprender programação não significa apenas “falar com máquinas”, mas desenvolver o chamado **pensamento computacional**: uma maneira organizada de resolver problemas grandes dividindo-os em partes menores.

Essa habilidade melhora:

- o raciocínio lógico;
- a criatividade;
- a organização;
- a capacidade de resolver problemas.

III. Fundamentos e Lógica (Aula 1)

III-A O que é um robô?

Um robô é uma máquina:

- autônoma;
- presente no mundo físico;
- capaz de perceber o ambiente;
- capaz de agir sobre o ambiente.

O Sparki utiliza sensores para coletar informações e motores para realizar ações.

Exemplo:

Um ar-condicionado liga e desliga sozinho, mas não toma decisões complexas sobre o ambiente físico como um robô faz.

III-B O que é um algoritmo?

Algoritmo é uma sequência lógica de passos usada para resolver um problema.

Exemplo: Escovar os dentes

1. Pegar a escova e a pasta.
2. Colocar pasta na escova.
3. Escovar os dentes.
4. Enxaguar a boca.

A ordem importa. Se inverter os passos, o algoritmo falha.

III-C As funções `setup()` e `loop()`

Todo programa do Sparki utiliza duas funções principais:

- **`setup()`**: executa apenas uma vez no início do programa;
- **`loop()`**: executa continuamente enquanto o robô estiver ligado.

III-D Estrutura básica do programa

```
#include <Sparki.h>

void setup() {
}

void loop() {
}
```

Listing 1. Estrutura básica do código

IV. Variáveis e Dados (Aula 2)

IV-A O conceito de variáveis

Imagine um armário cheio de gavetas.

Cada gaveta possui:

- um nome;
- um tipo de dado;
- um valor armazenado.

Na programação, essas gavetas são chamadas de **variáveis**.

Definição:

Variável é um espaço na memória usado para armazenar informações.

IV-B Tipos de dados

- **int**: números inteiros;
- **float**: números decimais;
- **char**: um único caractere;
- **bool**: verdadeiro ou falso.

```
int idade = 15;
float altura = 1.75;
char letra = 'A';
bool ligado = true;
```

Listing 2. Exemplo de variáveis

IV-C Operadores matemáticos

O Sparki frequentemente precisa realizar cálculos.

- Soma: +
- Subtração: -
- Multiplicação: *
- Divisão: /
- Resto da divisão: %

```
int a = 10;
int b = 3;

int soma = a + b; // resultado = 13
int resto = a % b; // resultado = 1
```

Listing 3. Exemplo de operações

IV-D Operadores de comparação

Atenção:

Para comparar igualdade usamos dois sinais: ==

- > maior que
- < menor que
- >= maior ou igual
- <= menor ou igual
- == igual
- != diferente

V. Condicionais e Decisão (Aula 3)

V-A if e else

As estruturas condicionais permitem que o robô tome decisões.

Exemplo do cotidiano:

“Se estiver chovendo, levo guarda-chuva. Senão, vou sem.”

```
if (distancia < 10) {
    // Acao verdadeira
} else {
    // Acao falsa
}
```

Listing 4. Estrutura condicional

V-B Operadores lógicos

- **&& (E)**: as duas condições devem ser verdadeiras;
- **|| (OU)**: apenas uma condição precisa ser verdadeira.

```
if (luz > 500 && distancia < 10) {
    sparki.beep();
}
```

Listing 5. Exemplo com operadores logicos

VI. Movimentação (Aula 4)

VI-A Motores

O Sparki possui, no total, 4 motores:

- (3) Motores de passo
 - Eles atuam nas rodas e nas garras do Sparki, possibilitando serem movidas com precisão.
 - O Sparki conta com 2 motores de passo, um para cada roda, podendo atuar com direções diferentes em cada uma a fim de rotacionar o robô, além de um para as garras, as possibilitando fechar e abrir.
- Servo motor
 - Controla a "cabeça" do Sparki independentemente do restante do robô, girando ela para os lados e mantendo o corpo estático.

Para usarmos os motores e seus respectivos componentes, podemos usar alguns comandos(funções) no código, vamos abordar as principais.

VI-B Funções de movimentação

O Sparki consegue calcular movimentos com precisão para diferentes direções(Forward, Backward, Right, Left)

- `sparki.moveForward(10);`
 - Faz o Sparki andar 10 cm para frente;
- `sparki.moveLeft(90);`
 - O Sparki faz uma curva de 90 graus para a esquerda.

Também é possível realizar movimentos utilizando o tempo, como andar para frente, esperar dois segundos andando, e parar:

```
sparki.moveForward();
delay(2000);
sparki.moveStop();
```

Listing 6. Movimento usando delay

Para movermos a "cabeça" do Sparki e suas garras, são utilizadas as seguintes funções:

- `sparki.servo(ângulo);`

- também existem alguns ângulos pré-definidos, como `SERVO_CENTER`, por exemplo, que faz o Sparki olhar para frente.
- `sparki.gripperOpen()`;
- `sparki.gripperClose()`;
- `sparki.gripperStop()`;
- Essas 3 funções são responsáveis pela garra, o `gripperStop()` faz com que o robô pare de executar o movimento de abrir ou fechar, sendo muito importante para evitar danos mecânicos no Sparki.

É possível utilizar as funções sem argumentos de ângulo ou distância, isso faz com que elas continuem a executar indefinidamente, até que um `sparki.moveStop()` ou um `sparki.gripperStop()` sejam acionados

VII. Repetição (Aula 5)

VII-A Laços de Repetição (**while**)

Em programação, muitas vezes precisamos repetir uma mesma ação várias vezes. Imagine ter que escrever o comando para o robô andar para frente 100 vezes. Isso seria cansativo e pouco eficiente.

Para resolver esse problema, usamos os **laços de repetição**.

Ideia principal:

Um laço de repetição executa um bloco de código várias vezes enquanto uma condição for verdadeira.

Exemplo do cotidiano:

“Enquanto o ônibus não chegar, continue esperando.”

Nesse caso:

- a condição é “o ônibus não chegou”;
- a ação repetida é “continuar esperando”.

Na programação, funciona da mesma forma.

1) Estrutura do while

```
while (condicao) {

    //Codigo que sera repetido

}
```

Listing 7. Estrutura basica do while

O computador verifica a condição:

- Se for **verdadeira**, o código executa;
- Quando virar **falsa**, o laço termina.

2) Exemplo simples

Neste exemplo, o Sparki irá emitir um som 5 vezes.

```
int contador = 0;

while (contador < 5) {

    sparki.beep();
```

```
    delay(500);

    contador = contador + 1;

}
```

Listing 8. Repetindo uma acao

Nesse exemplo:

- a variável `contador` começa valendo 0;
- o laço continua enquanto o contador for menor que 5;
- o Sparki faz um som;
- o programa espera meio segundo;
- o contador aumenta em 1;
- depois de 5 repetições, o laço termina.

3) Usando repetição no Sparki

Os laços são muito úteis em robótica, porque o robô precisa verificar sensores constantemente.

Exemplo:

O robô deve continuar andando até encontrar uma parede.

```
int distancia = sparki.ping();

while (distancia > 10) {

    sparki.moveForward();

    distancia = sparki.ping();

}

sparki.moveStop();
```

Listing 9. Sparki andando ate encontrar obstaculo

Passo a passo:

- 1) O sensor mede a distância;
- 2) Se a distância for maior que 10 cm, o robô anda;
- 3) O sensor mede novamente;
- 4) Quando a parede ficar próxima, o laço termina;
- 5) O robô para.

4) Cuidado com loops infinitos

Um erro muito comum é criar um laço que nunca termina.

```
while (true) {

    sparki.moveForward();

}
```

Listing 10. Exemplo de loop infinito

Nesse caso:

- a condição sempre será verdadeira;
- o robô nunca sairá do laço;
- ele continuará andando para sempre.

Por isso, é importante garantir que a condição possa mudar em algum momento.

5) *Resumo*

Os laços de repetição permitem:

- automatizar tarefas;
- evitar código repetido;
- criar comportamentos inteligentes;
- fazer o robô reagir continuamente ao ambiente.

VIII. Funções (Aula 7)

VIII-A O que são?

Uma função é um conjunto de passos que executa uma tarefa específica no programa. Uma função pode receber informações para auxiliar uma tarefa ou projeto, resolver algo mais complicado nesta função para ser usado no resto do programa.

VIII-B Como declarar funções?

Para declarar uma função, deve nomeá-la e em seguida definir seus parâmetros. Como por exemplo:

```
float calcularmedia(float v1, float v2)
{
    float media = (v1 + v2) / 2;
    return media;
}
```

Listing 11. Declarando uma função

Nesse exemplo, uma função `calcularmedia` é declarada, ela retorna um valor `float`, e usa como parâmetros os `floats` `v1` e `v2`. O que foi definido dentro dela é o que vai ser executado quando a função for chamada:

- 1) Ela cria uma variável chamada `media` dentro dela
- 2) Soma `v1 + v2` e divide o resultado por 2
- 3) Atribui o valor final para a variável `media`
- 4) Retorna essa variável para onde foi chamada

Lembrem-se: Funções não necessariamente precisam de parâmetros! `sparki.moveLeft()`, por exemplo, nada mais é que uma função pré-programada dentro da biblioteca `sparki`, que pode ser utilizada sem parâmetros!

VIII-C Chamar funções

Agora que declaramos a função, sempre que formos tirar uma média de dois números, basta chamarmos ela no código dando os parâmetros necessários, que no nosso caso, são dois números. É sempre bom, caso a função retorne um valor, o armazenar em uma variável, assim podemos usá-la em outra parte do código.

```
float media = calcularmedia(50, 250);
```

Listing 12. Chamando uma função

Nesse exemplo, os parâmetros enviados foram os números 50 e 250, e a nova variável `media` recebe o valor que a função calculou e retornou, nesse exemplo, 150.

IX. LCD (Aula 8)

IX-A O que é o LCD?

Uma das diversas funcionalidades do Sparki é o LCD, um pequeno monitor que pode mostrar informações como texto, números, formas geométricas, e, com um pouco de código, até mesmo animações!

IX-B Como usar o LCD

Primeiramente, vamos utilizar uma função muito importante chamada `sparki.clearLCD()`. Ela apaga o conteúdo que está presente na tela, então toda vez que formos começar a utilizar o painel ou atualizar algo na tela, devemos usá-la.

Agora, para conseguirmos escrever o texto na tela, utilizaremos o `sparki.print()`; ou a sua variação que pula automaticamente para a próxima linha ao final do texto, o `sparki.println()`; Isso vai fazer com que o argumento que você botar dentro da função seja escrito no LCD.

```
void setup() {
    sparki.clearLCD();
    sparki.print('Hello World!');
}
```

Listing 13. Algoritmo para escrever 'Hello World!'

Se você testou esse código, provavelmente viu que não apareceu nada na tela: isso ocorre pois devemos atualizar o LCD para que ele carregue o que foi definido, usando o `sparki.updateLCD()`; no fim do código.

```
void setup() {
    sparki.clearLCD();
    sparki.print("Hello World!");
    sparki.updateLCD();
}
```

Listing 14. Algoritmo corrigido e funcional

Não vamos entrar muito a fundo, mas existem também funções que conseguem desenhar formas geométricas, como `sparki.drawCircle()`; e a `sparki.drawLine()`; Usando elas e algumas outras, animações e desenhos podem ser feitos! Nesses casos, é importante adicionar um `delay()` no código, um intervalo de tempo, para dar tempo para o software trabalhar.

X. Sensor Ultrassônico (Aula 9)

X-A Sensores, o que são?

Vamos abordar 2 sensores principais responsáveis por captar informações do mundo real e traduzir para uma linguagem que ele entende. Nessa seção, vamos entrar a fundo no sensor Ultrassônico, responsável por medir distâncias.

X-B Funcionamento

O sensor ultrassônico, localizado na cabeça do Sparki, funciona emitindo ondas sonoras e medindo o tempo que elas demoram para ir, refletir em um objeto, e voltar. Com essa informação, ele consegue deduzir a distância que o objeto está do Sparki, se houver algum.

Para obtermos o valor dessa distância, usamos a função `sparki.ping()`; que retorna um valor numérico em centímetros, da distância do objeto mais próximo. Assim, podemos combinar essa informação com condicionais e assim alterar o comportamento do Sparki, sendo importante lembrar que essa função retorna -1 caso não haja objetos no alcance.

XI. Sensor Infravermelho (Aula 10)

O Sparki tem 2 componentes que envolvem ondas infravermelhas: os sensores de refletância, que identificam linhas e mudanças na superfície e o controle infravermelho, que pode ser usado para receber comandos de um controle, por exemplo.

XI-A Sensores de refletância

Contando com 5 deles na parte de baixo, eles funcionam soltando um raio infravermelho e recebendo ele de volta, analisando o quanto de luz foi absorvida pela superfície, e com base nisso determinando a cor dela. Seu principal uso é identificar uma fita preta, caminhando sobre ela no que é popularmente conhecido como seguidor de linhas.

```
void loop() {  
  int sensor = sparki.lineCenter();  
  if (sensor < 500) {  
    sparki.moveForward();  
  }  
}
```

Listing 15. seguidor de linhas(básico)

Nesse exemplo, o `sparki.lineCenter()`; retorna o valor do sensor central, e salva na variável `sensor`, e após isso compara com 500, um valor de exemplo para o valor que é retornado de uma superfície preta, como uma fita. Se o valor for menor que 500, assume-se que é uma linha preta, então ele segue reto.

XI-B Controle infravermelho

Além disso, o Sparki conta com um controle infravermelho que consegue se comunicar com o robô, enviando comandos. As ondas são enviadas em velocidades enormes, e o sensor, localizado na frente do Sparki, as recebe e as decodifica. Para ler elas no código, usamos a função `sparki.readIR()`; que retorna o valor numérico correspondente ao botão do controle.

XII. Conclusão

A robótica une programação, eletrônica e criatividade para resolver problemas reais.

Esperamos que esta apostila tenha sido um primeiro passo divertido e inspirador na sua jornada com tecnologia e engenharia.

**Bom aprendizado e divirta-se
programando!**